WebSocket Events: Server & Client Communication

Overview

WebSockets provide full-duplex communication channels over a single TCP connection. This guide explains the event-driven architecture of WebSocket communication between server and client.

Table of Contents

Server-Side Events

Using Node.js with `ws` library

```javascript
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });
```

Key Server Events

1. `connection`

Triggered when a client successfully connects to the server.

```javascript
wss.on('connection', (ws, request) => {
  console.log('New client connected');
  console.log('Client IP:', request.socket.remoteAddress);

  // ws is the WebSocket instance for this specific client
});
```

Triggered by: Client calling `new WebSocket('ws://localhost:8080')`

---

2. `message` (on individual client socket)

Triggered when the server receives a message from a specific client.

```javascript
wss.on('connection', (ws) => {
  ws.on('message', (data, isBinary) => {
    console.log('Received:', data.toString());

    // Echo back to client
    ws.send(`Server received: ${data}`);
  });
});
```

Triggered by: Client calling `ws.send('Hello Server')`

---

3. `close`

Triggered when a client disconnects.

```javascript
ws.on('close', (code, reason) => {
  console.log('Client disconnected');
  console.log('Code:', code);
  console.log('Reason:', reason.toString());
});
```

Triggered by:

- Client calling `ws.close()`
- Network failure
- Browser/tab closure

---

4. `error`

Triggered when an error occurs.

```javascript
ws.on('error', (error) => {
  console.error('WebSocket error:', error);
});
```

Triggered by: Connection errors, invalid frames, etc.

---

5. `ping` / `pong`

Used for keep-alive mechanisms.

```javascript
ws.on('ping', (data) => {
  console.log('Received ping from client');
});

ws.on('pong', (data) => {
  console.log('Received pong from client');
});
```

Triggered by: Client sending ping/pong frames

Client-Side Events

Browser WebSocket API

```javascript
const ws = new WebSocket('ws://localhost:8080');
```

Key Client Events

1. `open`

Triggered when connection is established.

```javascript
ws.addEventListener('open', (event) => {
  console.log('Connected to server');
  ws.send('Hello Server!');
});

// Or using onopen
ws.onopen = (event) => {
  console.log('Connection opened');
};
```

Triggered by: Successful handshake with server

---

2. `message`

Triggered when client receives a message from server.

```javascript
ws.addEventListener('message', (event) => {
  console.log('Message from server:', event.data);

  // Handle different data types
  if (typeof event.data === 'string') {
    console.log('Text message:', event.data);
  } else if (event.data instanceof Blob) {
    console.log('Binary message received');
  }
});

// Or using onmessage
ws.onmessage = (event) => {
  console.log('Received:', event.data);
};
```

Triggered by: Server calling `ws.send('message')`

---

3. `close`

Triggered when connection closes.

```javascript
ws.addEventListener('close', (event) => {
  console.log('Disconnected from server');
  console.log('Code:', event.code);
  console.log('Reason:', event.reason);
  console.log('Clean close:', event.wasClean);
});

// Or using onclose
ws.onclose = (event) => {
  console.log('Connection closed');
};
```

Triggered by:

- Server calling `ws.close()`
- Client calling `ws.close()`
- Network failure

4. `error`

Triggered when an error occurs.

```javascript
ws.addEventListener('error', (event) => {
  console.error('WebSocket error:', event);
});

// Or using onerror
ws.onerror = (event) => {
  console.error('Connection error');
};
```

Triggered by: Connection failures, invalid URLs, etc.

Event Flow & Triggering

Connection Establishment Flow

```
CLIENT                    SERVER
  |                         |
  |------ new WebSocket() ------->|
  |                         |
  |                  'connection' event
  |                         |
  |<----- Handshake Success ------|
  |                         |
'open' event                |
  |                         |
```

Message Exchange Flow

```
CLIENT                    SERVER
  |                         |
  |------ ws.send(data) --------->|
  |                         |
  |                  'message' event
  |                  ws.on('message')
  |                         |
  |<----- ws.send(reply) ---------|
  |                         |
```

```
'message' event                    |
ws.onmessage                       |
  |                                |
```

Disconnection Flow

```
CLIENT                          SERVER
  |                               |
  |------ ws.close() ------------>|
  |                               |
  |                        'close' event
  |                               |
  |<----- Close Ack --------------|
  |                               |
'close' event                     |
  |                               |
```

Complete Examples

Server Example (Node.js)

```javascript
```

```
const WebSocket = require('ws');

const wss = new WebSocket.Server({ port: 8080 });

console.log('WebSocket server running on ws://localhost:8080');

wss.on('connection', (ws, request) => {
  console.log('New client connected from:', request.socket.remoteAddress);

  // Send welcome message
  ws.send('Welcome to the WebSocket server!');

  // Handle incoming messages
  ws.on('message', (data, isBinary) => {
    const message = data.toString();
    console.log('Received from client:', message);

    // Echo back with timestamp
    ws.send(`Echo: ${message} (received at ${new Date().toISOString()})`);

    // Broadcast to all clients
    wss.clients.forEach((client) => {
      if (client !== ws && client.readyState === WebSocket.OPEN) {
        client.send(`Broadcast: ${message}`);
      }
    });
  });

  // Handle client disconnect
  ws.on('close', (code, reason) => {
    console.log('Client disconnected');
    console.log('Code:', code, 'Reason:', reason.toString());
  });

  // Handle errors
  ws.on('error', (error) => {
    console.error('WebSocket error:', error);
  });

  // Handle ping/pong for keep-alive
  ws.on('ping', () => {
    console.log('Received ping from client');
  });

  ws.on('pong', () => {
    console.log('Received pong from client');
```

```
  });
});

// Server-level error handling
wss.on('error', (error) => {
  console.error('Server error:', error);
});
```

Client Example (Browser)

javascript

```
// Create WebSocket connection
const ws = new WebSocket('ws://localhost:8080');

// Connection opened
ws.addEventListener('open', (event) => {
  console.log('Connected to WebSocket server');

  // Send initial message
  ws.send('Hello from client!');
});

// Listen for messages from server
ws.addEventListener('message', (event) => {
  console.log('Message from server:', event.data);

  // Display in UI
  const messageDiv = document.createElement('div');
  messageDiv.textContent = event.data;
  document.getElementById('messages').appendChild(messageDiv);
});

// Handle connection close
ws.addEventListener('close', (event) => {
  console.log('Disconnected from server');
  console.log('Code:', event.code);
  console.log('Reason:', event.reason);
  console.log('Was clean:', event.wasClean);

  // Attempt reconnection after 3 seconds
  setTimeout(() => {
    console.log('Attempting to reconnect...');
    // Reinitialize connection
  }, 3000);
});

// Handle errors
ws.addEventListener('error', (event) => {
  console.error('WebSocket error:', event);
});

// Function to send messages
function sendMessage(message) {
  if (ws.readyState === WebSocket.OPEN) {
    ws.send(message);
  } else {
    console.error('WebSocket is not open. Ready state:', ws.readyState);
```

```
  }
}

// Example: Send message on button click
document.getElementById('sendBtn').addEventListener('click', () => {
  const input = document.getElementById('messageInput');
  sendMessage(input.value);
  input.value = '';
});

// Close connection on page unload
window.addEventListener('beforeunload', () => {
  ws.close(1000, 'Client closing');
});
```

HTML Client Example

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>WebSocket Client</title>
</head>
<body>
  <h1>WebSocket Chat</h1>

  <div id="status">Disconnected</div>

  <div id="messages" style="border: 1px solid #ccc; height: 300px; overflow-y: auto; padding: 10px;">
  </div>

  <input type="text" id="messageInput" placeholder="Type a message...">
  <button id="sendBtn">Send</button>
  <button id="disconnectBtn">Disconnect</button>

  <script src="client.js"></script>
</body>
</html>
```

WebSocket Ready States

```javascript
WebSocket.CONNECTING  // 0 - Connection not yet established
WebSocket.OPEN        // 1 - Connection is open and ready
WebSocket.CLOSING     // 2 - Connection is closing
WebSocket.CLOSED      // 3 - Connection is closed
```

Checking Connection State

```javascript
// Client-side
if (ws.readyState === WebSocket.OPEN) {
  ws.send('Message');
}

// Server-side (using ws library)
if (ws.readyState === WebSocket.OPEN) {
  ws.send('Message');
}
```

Common Close Codes

Code	Meaning	Description
1000	Normal Closure	Successful operation / regular socket shutdown
1001	Going Away	Endpoint is going away (e.g., server down or browser navigating away)
1002	Protocol Error	Endpoint terminating due to protocol error
1003	Unsupported Data	Connection terminated due to unsupported data type
1006	Abnormal Closure	No close frame received (connection lost)
1007	Invalid Frame	Invalid data received (e.g., non-UTF-8 in text message)
1008	Policy Violation	Message violates policy
1009	Message Too Big	Message too large to process
1011	Internal Error	Server encountered an unexpected condition

---

Best Practices

1. Always Handle All Events

javascript

```
// Client
ws.onopen = handleOpen;
ws.onmessage = handleMessage;
ws.onerror = handleError;
ws.onclose = handleClose;
```

2. Implement Reconnection Logic

javascript

```
let reconnectAttempts = 0;
const maxReconnectAttempts = 5;

function connect() {
  const ws = new WebSocket('ws://localhost:8080');

  ws.onclose = () => {
    if (reconnectAttempts < maxReconnectAttempts) {
      reconnectAttempts++;
      setTimeout(connect, 1000 * reconnectAttempts);
    }
  };

  ws.onopen = () => {
    reconnectAttempts = 0;
  };
}
```

3. Implement Heartbeat/Ping-Pong

javascript

```
// Server-side
const interval = setInterval(() => {
  wss.clients.forEach((ws) => {
    if (ws.isAlive === false) return ws.terminate();

    ws.isAlive = false;
    ws.ping();
  });
}, 30000);

wss.on('connection', (ws) => {
  ws.isAlive = true;
  ws.on('pong', () => {
    ws.isAlive = true;
  });
});
```

4. Validate and Sanitize Messages

javascript

```
ws.on('message', (data) => {
  try {
    const message = JSON.parse(data.toString());

    // Validate message structure
    if (!message.type || !message.payload) {
      ws.send(JSON.stringify({ error: 'Invalid message format' }));
      return;
    }

    // Process valid message
    handleMessage(message);
  } catch (error) {
    ws.send(JSON.stringify({ error: 'Invalid JSON' }));
  }
});
```

5. Handle Binary Data

javascript

```
// Server receiving binary
ws.on('message', (data, isBinary) => {
  if (isBinary) {
    console.log('Received binary data:', data.length, 'bytes');
    // Process binary data
  } else {
    console.log('Received text:', data.toString());
  }
});

// Client sending binary
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);
ws.send(view);
```

Debugging Tips

1. Log All Events

javascript

```
const events = ['open', 'message', 'close', 'error'];
events.forEach(event => {
  ws.addEventListener(event, (e) => {
    console.log(`[${new Date().toISOString()}] ${event}:`, e);
  });
});
```

2. Use Browser DevTools

- Open DevTools → Network tab
- Filter by "WS" to see WebSocket connections
- Click on connection to see frames exchanged

3. Monitor Connection State

javascript

```
setInterval(() => {
  console.log('WebSocket state:', ws.readyState);
}, 5000);
```

Summary

Server-side events are triggered by client actions:

- `connection` ← Client connects
- `message` ← Client sends data
- `close` ← Client disconnects
- `error` ← Connection issues

Client-side events are triggered by server actions:

- `open` ← Server accepts connection
- `message` ← Server sends data
- `close` ← Server closes connection
- `error` ← Connection issues

The event-driven model allows for real-time, bidirectional communication perfect for chat applications, live updates, gaming, and collaborative tools.

Additional Resources

- MDN WebSocket API
- ws Library Documentation
- WebSocket Protocol RFC 6455